Segundo avance del proyecto: incremento funcional de PedidosFlow

Gemas de Cristal

Steven Suaza Montiel e Isabel Cristina Duque Cano

Politécnico Grancolombiano

Práctica Aplicada de Sistemas

Neider Xavier Vergara

22 de octubre de 2026

Introducción

Este documento presenta el segundo avance académico de PedidosFlow y la evolución desde el primer incremento funcional hacia una versión que refuerza la persistencia, flexibiliza la conversación y organiza las excepciones para revisión humana. La aplicación conserva el dominio de tortas personalizadas, el estilo visual aprobado y el uso de una interfaz web que simula la conversación de WhatsApp.

La solución atiende un problema operativo: después de confirmar un pedido personalizado, el cliente puede solicitar cambios de cantidad, diseño o fecha. El encargado necesita decidir si el cambio puede aplicarse según la etapa de producción y conservar el registro de lo solicitado y de la decisión. La producción personalizada exige equilibrar requerimientos, capacidad y tiempos disponibles (Zhang & Tseng, 2009), y convertir la voz del cliente en especificaciones que el proceso pueda atender (Wang & Tseng, 2014).

La presentación del informe sigue las pautas generales de formato para trabajos estudiantiles de APA, séptima edición (American Psychological Association [APA], 2020).

El informe mantiene los ocho componentes solicitados por la guía docente. Se distingue entre funciones incorporadas al paquete ejecutable, resultados históricos de la versión base y verificaciones de la versión final que deben quedar respaldadas con salidas de consola o capturas reales. Las áreas marcadas como evidencia visual indican el lugar donde el equipo debe insertar capturas tomadas de la aplicación y del repositorio; no representan resultados simulados.

1. Actualización del primer avance

1.1 Problema y delimitación

Los negocios que elaboran tortas personalizadas por encargo reciben solicitudes de cambio después de confirmar el pedido. Cuando las conversaciones se mezclan con otros mensajes y la versión vigente del pedido no se registra de forma clara, el negocio debe

reconstruir qué pidió el cliente y decidir con información dispersa si el cambio todavía es posible. Esto puede producir errores de coordinación, compromisos de entrega inviables y falta de trazabilidad.

Frente al alcance amplio del primer avance (tortas, camisetas, muebles y arreglos florales), el sistema se acota a tortas personalizadas. En este dominio, la cantidad y la fecha son atributos que pueden comprobarse con reglas por etapa. Un cambio de diseño puede requerir criterio humano porque una imagen de referencia no garantiza que el resultado sea técnicamente elaborable. Por ello, los cambios automáticos y los que necesitan revisión se tratan de manera diferente.

El supuesto de que las reglas son adecuadas para un negocio real todavía requiere validación con una persona encargada de producción. Las reglas incorporadas son configurables y sirven como punto de partida académico; no deben presentarse como políticas universales para todas las pastelerías.

1.2 Justificación

Guardar solo el último estado no permite conocer las versiones anteriores ni justificar una decisión. PedidosFlow conserva el pedido, los cambios, sus resultados y las decisiones manuales. Esta trazabilidad es relevante para la producción personalizada, en la que los requerimientos del cliente se deben coordinar con los recursos y los tiempos del proceso (Zhang & Tseng, 2009; Wang & Tseng, 2014).

La segunda iteración atiende tres problemas concretos: la ruta de almacenamiento podía confundirse si el servidor se iniciaba desde distintas carpetas; el chat dependía demasiado de botones y respuestas predefinidas; y las decisiones humanas no debían comunicarse como definitivas antes de guardarse. La versión actual resuelve estos puntos con ruta de datos estable, entrada libre y reconocimiento acotado de intenciones, menú compacto

separado del área de escritura y mensaje final en el chat web después de que el encargado aprueba o rechaza la solicitud.

1.2.1 Evidencia documental de negocios reales

En Colombia, Galvis Gómez et al. (2020) caracterizaron diez emprendimientos de Bucaramanga y su área metropolitana. En esa muestra, el 30 % informó utilizar WhatsApp Business y el 70 % no; además, la mitad de las iniciativas correspondía a alimentos y bebidas. El estudio se desarrolló durante la pandemia de COVID-19 y la muestra es pequeña, por lo que estos porcentajes no representan el mercado colombiano actual ni específicamente a las pastelerías. Sin embargo, ofrecen evidencia contextual obtenida de emprendimientos reales de que las prácticas digitales y los recursos tecnológicos no son homogéneos.

En un estudio cualitativo con entrevistas semiestructuradas a doce micro y pequeñas empresas comerciales de Gauteng, Sudáfrica, Azionya y Mashigo (2025) identificaron como factores relevantes de la comunicación empresarial por WhatsApp la claridad de los objetivos, los mensajes conversacionales y personalizados, el uso estratégico de las funciones de la plataforma y la medición de resultados. Este hallazgo respalda el diseño de una conversación comprensible y contextualizada en PedidosFlow, aunque no determina por sí mismo las reglas productivas específicas de una pastelería.

Como caso empresarial descriptivo, Wassenger (2025) presenta a Bella’s Bakeshop, una pequeña panadería que gestionaba por WhatsApp pedidos, confirmaciones y actualizaciones de entrega, y que incorporó automatizaciones para estos mensajes. Esta fuente procede de un proveedor de soluciones de mensajería; por tanto, se utiliza como ejemplo de práctica reportada, no como evaluación independiente ni como evidencia de que los mismos resultados se producirían en otro negocio.

En conjunto, estas fuentes amplían la justificación con datos de emprendimientos encuestados, entrevistas a pequeños comercios y un caso de panadería. Constituyen evidencia

documental sobre necesidades de comunicación y organización de pedidos en negocios reales y ofrecen antecedentes pertinentes para el diseño de PedidosFlow. Debido a que los estudios provienen de muestras y contextos distintos, las reglas de capacidad diaria, fechas y etapa de producción se mantienen configurables para ajustarlas a las prácticas de una pastelería colombiana.

1.3 Objetivos

Objetivo general. Construir y verificar un incremento funcional de PedidosFlow que registre y evalúe cambios de pedidos de tortas personalizadas, preserve los datos entre ejecuciones y ofrezca un chat web más flexible, manteniendo la trazabilidad y separando las decisiones automáticas de las que requieren revisión humana.

Objetivos específicos:

- Mantener una matriz de reglas editable que relacione la etapa de producción con los cambios de cantidad, diseño y fecha que pueden aplicarse o deben revisarse.
- Conservar pedidos, cambios, notificaciones y configuración mediante archivos JSON en una ruta de datos estable.
- Permitir que el chat interprete intenciones comunes escritas por el usuario y solicite el dato faltante cuando sea necesario.
- Validar fechas, horas y confirmaciones antes de enviar una operación al servidor o crear un pedido.
- Verificar la ruta de persistencia, los cambios reales de pedidos, los flujos de revisión y el nuevo comportamiento conversacional con pruebas automatizadas y manuales. Los resultados históricos se presentan como referencia; los de la versión actual deben confirmarse con la ejecución final y las evidencias de la entrega.

1.4 Marco teórico inicial

WhatsApp se utiliza como canal de comunicación entre pequeños negocios y sus clientes. En el proyecto, esta referencia justifica el estilo conversacional, pero no significa que exista una integración con la plataforma de WhatsApp Business (Azionya & Mashigo, 2025; Meta, 2025). El canal real queda fuera del alcance técnico actual.

Scrum organiza el trabajo en iteraciones y define un incremento utilizable que cumple una Definition of Done acordada (Schwaber & Sutherland, 2020). El equipo adapta la planificación a dos integrantes, utiliza un backlog priorizado y propone revisar los cambios mediante pull requests. Las historias de usuario describen la necesidad desde la perspectiva de quien usa el producto; sus criterios de aceptación convierten esa necesidad en condiciones verificables (Cohn, 2004).

El principio técnico aplicado es la validación basada en reglas explícitas: la decisión se determina con datos del pedido, su etapa y una matriz editable, en lugar de dispersar las condiciones en el código de la interfaz. Las solicitudes ambiguas no se resuelven inventando datos: el chat debe solicitar una aclaración o dirigir el caso al encargado.

1.5 Propuesta de solución

PedidosFlow contiene dos vistas conectadas a un servidor Express: el chat web simulado para el cliente y la plataforma de gestión para el encargado. La API valida las entradas, consulta el pedido, evalúa la regla de la etapa y escribe los cambios en archivos JSON. La interfaz administrativa permite consultar pedidos, producción, clientes, cambios pendientes, historial, notificaciones y configuración.

La solución conserva el diseño de referencia y añade comportamiento comprobable: los pedidos, cambios, notificaciones y reglas se guardan en JSON; después de una operación la interfaz vuelve a consultar el estado del servidor; las solicitudes que requieren criterio humano no se aplican hasta que el encargado decide. El chat web puede mostrar el resultado de esa

decisión, pero el envío por WhatsApp real requiere una integración externa que no forma parte de este incremento.

1.6 Metodología ágil

El equipo adopta iteraciones de dos semanas, priorización del backlog, seguimiento con GitHub Projects y revisión del trabajo antes de integrarlo a la rama principal. La capacidad documentada es de 4 horas semanales para Steven y 6 para Isabel. En dos semanas son 20 horas brutas; al reservar 25 % para integración e imprevistos, la capacidad efectiva estimada es de 15 horas. La estimación de tareas de este incremento se presenta como planificación y debe contrastarse con el tiempo realmente registrado por el equipo. Esta práctica es consistente con un flujo de trabajo ligero basado en ramas y pull requests, como el documentado por GitHub (s. f.).

1.7 Roles y responsabilidades

Tabla 1

Roles y responsabilidades del equipo

Integrante	Rol acordado	**Responsabilidades**	**Disponibilidad**
Isabel Cristina Duque Cano	Product Owner y desarrolladora	Priorizar historias, definir criterios de aceptación, desarrollar API e interfaces, mantener el repositorio y documentar decisiones técnicas.	6 h/semana
Steven Suaza Montiel	Scrum Master y desarrollador	Facilitar planeación y seguimiento, revisar pull requests, preparar y ejecutar pruebas, apoyar documentación y evidencias.	4 h/semana

Nota. La distribución de roles y la disponibilidad semanal fueron confirmadas por el equipo para esta iteración.

2. Product Backlog actualizado

El Product Backlog separa las historias que forman parte de la aplicación actual de las mejoras y dependencias que quedan para una iteración posterior. En los estados se diferencia

la implementación en el paquete de la verificación final: una función incorporada no se declara probada si no hay una ejecución o evidencia que lo demuestre.

Tabla 2

Backlog del producto e incremento 2

ID	Historia y criterio de aceptación resumido	Prioridad / estimación	Estado	Responsable
PF-01	Como encargado, quiero consultar pedidos con cliente, etapa y avance. CA: datos corresponden al almacenamiento.	Media; 2 pt	Implementada*	Isabel
PF-02	Como cliente, quiero cambiar cantidad según la etapa. CA: validar pedido, cantidad, decisión y motivo; registrar historial.	Alta; 5 pt	Implementada*	Isabel
PF-03	Como cliente, quiero enviar un diseño o foto para revisión. CA: crear solicitud y mostrar imagen al encargado.	Alta; 5 pt	Implementada*	Isabel
PF-04	Como encargado, quiero aprobar o mantener el rechazo. CA: actualizar solo si se aprueba y guardar decisión manual.	Alta; 3 pt	Implementada*	Steven
PF-05	Como encargado, quiero actualizar los 7 pasos de producción. CA: recalcular etapa y porcentaje desde el checklist.	Alta; 3 pt	Implementada*	Isabel
PF-06	Como encargado, quiero consultar el historial. CA: mostrar tipo, resultado, motivo y fecha; persistir tras reinicio.	Alta; 3 pt	Implementada*	Steven
PF-07	Como cliente, quiero solicitar una fecha nueva. CA: validar formato y regla por etapa; registrar resultado.	Media; 3 pt	Implementada*	Isabel
PF-08	Como dueño, quiero editar reglas y datos del negocio. CA: persistir la configuración y aplicarla en validaciones siguientes.	Media; 3 pt	Implementada*	Isabel
PF-09	Como encargado, quiero consultar clientes y sus pedidos. CA: agrupar pedidos del mismo teléfono sin duplicar cliente.	Baja; 2 pt	Implementada*	Isabel
PF-10	Como cliente, quiero realizar solicitudes por WhatsApp real. CA: recibir mensaje y devolver respuesta por WhatsApp Business.	Alta; 8 pt	Backlog posterior: integración externa	Steven

ID	Historia y criterio de aceptación resumido	Prioridad / estimación	Estado	Responsable
PF-11	Como cliente, quiero recibir en el chat web la decisión final después de la revisión humana. CA: el chat no anticipa la decisión; tras guardarla, informa aprobación o rechazo y muestra los datos vigentes.	Media; 3 pt	Implementada en chat web simulado	Isabel
PF-12	Como encargado, quiero iniciar sesión. CA: impedir acceso administrativo sin credenciales válidas.	Media; 5 pt	Backlog posterior: autenticación	Steven
PF-13	Como dueño, quiero ver estadísticas de cambios. CA: reportar cantidades por tipo y resultado.	Baja; 5 pt	Backlog posterior: analítica	Steven e Isabel
PF2-01	Como encargado, quiero que los datos usen una ruta estable. CA: ruta independiente de la terminal y visible al iniciar.	Alta; 2 h	Implementada†	Isabel
PF2-02	Como sistema, quiero guardar JSON de forma más segura. CA: escritura temporal, reemplazo y recuperación ante error.	Alta; 2 h	Implementada†	Isabel
PF2-03	Como cliente, quiero escribir mensajes además de pulsar botones. CA: entrada libre en los pasos conversacionales.	Alta; 2 h	Implementada†	Isabel
PF2-04	Como cliente, quiero que se interpreten intenciones comunes. CA: consultar pedido, crear pedido y pedir cambios frecuentes.	Alta; 1 h	Implementada†	Isabel
PF2-05	Como cliente, quiero introducir fechas y horas cotidianas. CA: reconocer formatos conocidos y pedir corrección si son inválidos.	Media; 1.5 h	Implementada†	Isabel
PF2-06	Como cliente, quiero confirmar explícitamente un pedido nuevo. CA: un “no” o respuesta ambigua no crea el pedido.	Alta; 1 h	Implementada†	Isabel / Steven
PF2-07	Como cliente, quiero conservar la conversación al recargar la pestaña. CA: restaurar transcripción limitada sin guardar base64 de imágenes.	Media; 1 h	Implementada†	Steven

ID	Historia y criterio de aceptación resumido	Prioridad / estimación	Estado	Responsable
PF2-08	Como cliente, quiero enviar en una sola frase el pedido, el cambio y el nuevo valor. CA: el chat aprovecha los datos reconocidos y pregunta solo por los que falten.	Alta; 2 h	Incluida en el paquete; regresión por evidenciar	Isabel
PF2-09	Como cliente, quiero describir un diseño fuera del catálogo sin imagen. CA: la propuesta se guarda como pendiente y no cambia el diseño vigente hasta la decisión humana.	Alta; 2 h	Incluida en el paquete; regresión por evidenciar	Isabel
PF2-10	Como encargado, quiero que una fecha pasada válida pase a revisión. CA: la fecha original no cambia antes de la decisión humana.	Alta; 1 h	Incluida en el paquete; regresión por evidenciar	Isabel
PF2-11	Como encargado, quiero definir el máximo de pedidos activos por día. CA: una fecha que alcanza el límite genera una solicitud pendiente sin modificar el pedido original.	Alta; 1 h	Incluida en el paquete; regresión por evidenciar	Isabel
PF2-12	Como equipo, queremos datos de demostración iniciales para probar el programa de inmediato. CA: aparecen cinco pedidos en una carpeta nueva; no se duplican ni reinician en ejecuciones siguientes.	Media; 1 h	Incluida en el paquete; regresión por evidenciar	Isabel

ID	Historia y criterio de aceptación resumido	Prioridad / estimación	Estado	Responsable
PF2-13	Como cliente, quiero distinguir las opciones rápidas del campo de escritura. CA: las opciones se muestran en un panel compacto bajo la conversación y “Volver al menú” queda dentro del mismo panel.	Media; esfuerzo adicional no registrado en la planeación inicial	Incluida en el ZIP final; captura por insertar	Isabel
PF2-14	Como cliente, quiero conocer la decisión de una solicitud pendiente solo después de que el encargado la guarde. CA: al crearla se informa que está pendiente; después de aprobar/rechazar se muestra el resultado y el pedido vigente.	Alta; esfuerzo adicional no registrado en la planeación inicial	Incluida en el ZIP final; E2E por evidenciar	Isabel / Steven
PF2-15	Como equipo, queremos contrastar las reglas de etapa, fecha y capacidad con una pastelería real. CA: documentar la consulta y los hallazgos, y ajustar o justificar las reglas con evidencia.	Alta; 3 h por estimar tras el contacto	Ampliar el contraste operativo local; issue #27	Steven e Isabel

Nota. “Implementada en el paquete” indica que el código está incluido en la versión entregada. “Regresión por evidenciar” indica que se debe adjuntar el resultado real de ejecutar la verificación final en el equipo del proyecto. La conexión real con WhatsApp, la autenticación y los reportes analíticos permanecen fuera del alcance de esta iteración.

2.1 Criterios de aceptación por flujo

Tabla 3

Criterios de aceptación agrupados por flujo

Flujo	Criterio verificable
Pedido inexistente	Cuando se escribe un identificador que no existe, el chat informa que no lo encontró y permite corregirlo sin pedir datos del cambio.
Cambio de cantidad o fecha	Cuando el pedido existe, el servidor deriva la etapa de producción, consulta la regla correspondiente, aplica o rechaza el cambio o lo deja en revisión, y registra el resultado.
Cambio de diseño personalizado	Cuando se envía una referencia gráfica que requiere criterio humano, la solicitud queda pendiente; el pedido no cambia hasta que el encargado tome una decisión.
Producción e historial	Cuando cambia el checklist, la plataforma actualiza etapa y avance. Cada cambio conserva tipo, valores, fecha y decisiones automáticas/manuales.
Configuración	Cuando se modifica una regla válida, las validaciones siguientes usan la nueva configuración; una configuración inválida no debe reemplazar la vigente.
Persistencia tras reinicio	Después de crear o modificar un pedido y reiniciar el servidor, el pedido, el cambio y la configuración deben seguir disponibles.
Lenguaje natural y confirmación	Cuando el cliente escribe una intención común, el chat inicia el flujo correspondiente. Cuando responde negativamente al resumen de un pedido nuevo, no debe crearlo.
Continuidad de chat	Cuando se recarga la misma pestaña, la transcripción reciente se restaura; esta continuidad no sustituye la persistencia del servidor.
Menú de opciones y escritura libre	Cuando el cliente observa el chat, las opciones rápidas aparecen en un panel compacto bajo los mensajes y separado del área de escritura; “Volver al menú” queda en el panel.
Aviso después de la revisión humana	Cuando una solicitud pasa a pendientes, el cliente recibe solo un aviso de espera. Después de que el encargado guarda la decisión, el chat informa aprobación o rechazo y muestra el estado real del pedido.
Precarga de demostración	En una carpeta nueva o vacía, el sistema carga los pedidos #152–#156 y la solicitud pendiente de diseño; al reiniciar conserva los cambios y no duplica pedidos.

3. Planeación de la iteración

3.1 Objetivo de la iteración

Mejorar la fiabilidad del almacenamiento y la usabilidad del chat web simulado; permitir solicitudes redactadas con palabras propias; dirigir fechas pasadas, días que alcanzaron la

capacidad configurada y diseños fuera del catálogo a revisión; y mostrar al cliente la decisión humana solamente después de que se guarda en el panel del encargado.

3.2 Historias seleccionadas y justificación

La planeación original seleccionó PF2-01 a PF2-07. Durante el cierre se incorporaron correcciones complementarias para completar el flujo conversacional: lectura de solicitudes completas, diseño personalizado descrito por texto, manejo de fecha pasada y capacidad diaria, precarga estable de datos de demostración, menú compacto y aviso posterior a la revisión. Estas ampliaciones se documentan como alcance adicional; no se reconstruyen horas reales que no estén registradas.

3.3 Tareas técnicas, estimación y distribución

Tabla 4

Plan técnico propuesto para una iteración de 15 horas efectivas

ID	Tarea técnica	Historias	Horas	Responsable
T1	Resolver la ruta de la carpeta de datos y de archivos estáticos desde la raíz del proyecto.	PF2-01	2.0	Isabel
T2	Reforzar escritura JSON con temporal único y recuperación ante fallo de reemplazo.	PF2-02	2.0	Isabel
T3	Incorporar campo de escritura y reconocimiento de intenciones frecuentes.	PF2-03, PF2-04	3.0	Isabel
T4	Interpretar y validar fechas/horas habituales antes de enviar la solicitud.	PF2-05	1.5	Isabel
T5	Verificar confirmaciones negativas y conservar transcripción en sessionStorage.	PF2-06, PF2-07	1.5	Isabel (0.5) y Steven (1.0)
T6	Agregar escenarios E2E de intención, consulta, recarga y no-confirmación.	PF2-03 a PF2-07	2.5	Steven
T7	Ejecutar regresión, actualizar documentación y hacer revisión cruzada.	Todas	2.5	Steven
Total			15.0	Isabel: 9 h; Steven: 6 h

Nota. La tabla conserva la capacidad estimada documentada del equipo; se presenta como plan de distribución y no como registro comprobado de horas efectivamente trabajadas.

3.4 Riesgos y dependencias

Tabla 5

Riesgos y mitigaciones

ID	Riesgo o dependencia	Probabilidad / impacto	Mitigación
R1	Instalación incompleta de dependencias o falta de conectividad para npm.	Alta / Alto	Ejecutar npm ci en el equipo con acceso a npm; registrar salida y versiones.
R2	Los datos históricos pueden existir en una carpeta distinta a la nueva ruta.	Media / Alto	Verificar la ruta impresa por el servidor y configurar PEDIDOSFLOW_DATOS antes de eliminar o migrar archivos.
R3	La interpretación basada en frases puede fallar con mensajes ambiguos.	Media / Medio	Solicitar aclaración y probar variantes comunes; no inventar cantidades ni fechas.
R4	La revisión de las nuevas funciones depende de ejecutar E2E y pruebas de regresión.	Alta / Alto	Instalar dependencias, ejecutar npm run verify y el recorrido E2E, guardar las salidas y revisar entre integrantes.
R5	La API oficial de WhatsApp requiere configuración externa.	Alta / Medio	Mantener el chat web como simulación y planificar la integración en una historia posterior.
R6	La matriz no está validada todavía con una pastelería real.	Media / Alto	Realizar prueba con una persona encargada y ajustar reglas según el proceso real.

4. Prototipo actualizado

El prototipo ejecutable del incremento es la interfaz web conectada a una API Express. El prototipo inicial de Figma Make se conserva como referencia visual; las vistas del paquete ejecutable consultan y actualizan los datos del servidor. En el chat, los mensajes y las opciones rápidas aparecen en zonas distintas: el menú compacto está debajo de la conversación y encima del campo de escritura.

4.1 Flujo principal

El cliente abre el chat web y selecciona una acción o escribe una solicitud común.

El chat identifica la intención y extrae el número del pedido, si aparece. Si falta un dato, pregunta por ese dato concreto.

El servidor comprueba que el pedido exista y valida la entrada antes de procesarla.

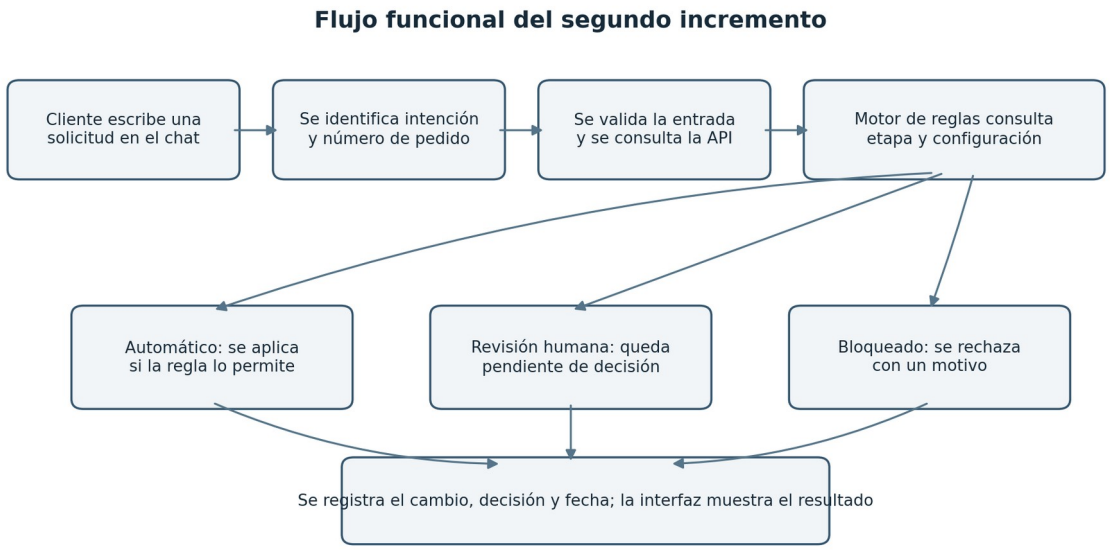
Para cantidad y fecha, el motor aplica la matriz de reglas según la etapa. Un cambio de diseño que requiere criterio queda pendiente para el encargado.

El servidor guarda el cambio automático o la solicitud pendiente. Cuando requiere revisión humana, el chat inicialmente informa que la solicitud está pendiente. Solo cuando el encargado guarda la aprobación o el rechazo, la conversación muestra el aviso final y consulta de nuevo el pedido para presentar los datos vigentes; en caso de rechazo se conservan los valores anteriores.

Para registrar un pedido nuevo, el chat presenta un resumen y requiere una confirmación afirmativa explícita. Una negativa o respuesta ambigua no crea el pedido. Por defecto, una carpeta de datos nueva se inicializa con cinco pedidos de demostración (#152 a #156); las ejecuciones siguientes conservan las modificaciones y no duplican el conjunto de muestra.

Figura 1

Flujo funcional del segundo incremento



Nota. La aplicación simula el canal WhatsApp en una interfaz web; no envía mensajes a WhatsApp real. La decisión automática se rige por la matriz configurable y el estado de

producción. El mensaje definitivo de una solicitud en revisión aparece únicamente después de que el encargado guarda su decisión.

4.2 Pantallas, navegación y datos

Tabla 6

Vistas de la interfaz y comportamiento esperado

Pantalla	Datos solicitados o mostrados	Respuesta esperada
Chat del cliente	Número de pedido, acción, cantidad, diseño/foto, fecha y hora.	Respuesta en lenguaje cotidiano; pregunta por el dato faltante, confirma, rechaza con motivo o informa que queda en revisión.
Pedidos	Identificador, cliente, cantidad, diseño, fecha, etapa y avance.	Consulta del estado guardado y acceso al detalle.
Cambios pendientes	Cambio solicitado, valores anterior/nuevo, motivo y referencia gráfica cuando corresponde.	Permite al encargado aprobar o mantener el rechazo y guarda la decisión.
Producción	Checklist de siete pasos por pedido.	Actualiza etapa, porcentaje y estado a partir del checklist.
Clientes	Información del cliente y pedidos asociados.	Agrupa pedidos por teléfono para reconocer clientes recurrentes.
Historial y notificaciones	Cambios, decisión, fecha, responsable y eventos.	Permite revisar la trazabilidad y marcar notificaciones como leídas.
Configuración	Matriz de reglas, catálogo, etapas y datos del negocio.	Guarda reglas y las aplica en validaciones posteriores.

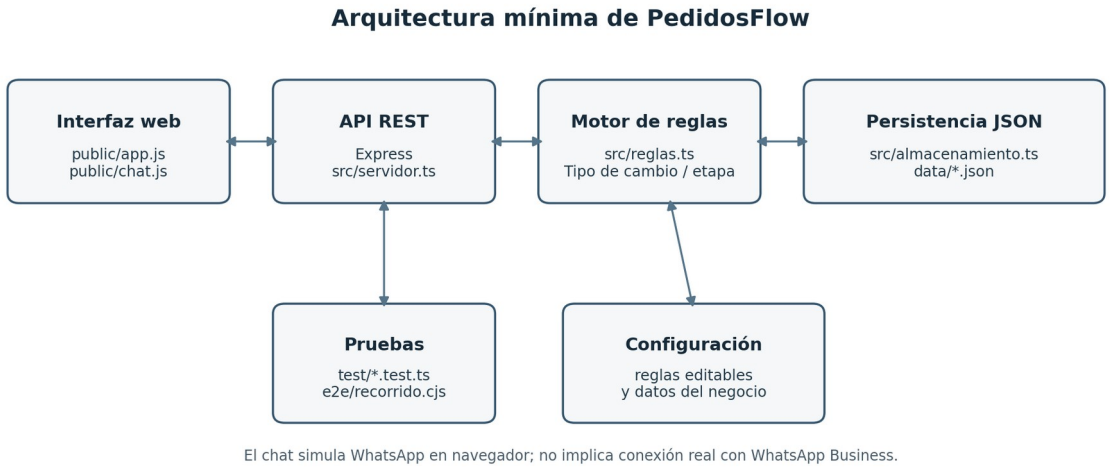
4.3 Conservación del diseño

La actualización visual mantiene colores, navegación, tarjetas y estructura principal. El menú de opciones se presenta en una franja compacta debajo de los mensajes y separada del área donde el cliente escribe. «Volver al menú» forma parte de esa misma franja para que no parezca un control ajeno a la conversación.

5. Diseño técnico mínimo

Figura 2

Arquitectura mínima de PedidosFlow



Nota. La interfaz cliente/encargado consume la API REST. La persistencia se hace en archivos JSON locales; las pruebas verifican por separado reglas, API, persistencia y recorridos de navegador.

5.1 Componentes y responsabilidades

Tabla 7

Componentes del sistema

Componente	Archivos	Responsabilidad
Interfaz del encargado	public/app.js	Muestra pedidos, detalle, cambios pendientes, producción, clientes, historial, notificaciones y configuración.
Chat del cliente	public/chat.js	Guía la conversación, acepta texto libre, reconoce intenciones comunes y muestra la respuesta.
API REST	src/servidor.ts	Valida solicitudes HTTP, consulta el pedido, coordina el motor de reglas y persiste resultados.
Motor de reglas	src/reglas.ts	Deriva la etapa, valida datos y aplica reglas según tipo de cambio y etapa.
Almacenamiento	src/almacenamiento.ts	Lee y escribe JSON en la carpeta de datos definida y crea datos de demostración si corresponde.
Pruebas	test/*.test.ts y e2e/recorrido.cjs	Verifican reglas, API, persistencia y recorrido de la interfaz.

5.2 Flujo de datos

El chat o la plataforma envían una solicitud a un endpoint de la API.

La API valida el esquema y comprueba la existencia y el estado del pedido.

El motor deriva la etapa a partir del checklist y consulta la configuración guardada.

La decisión aplica el cambio, lo deja pendiente de revisión o lo rechaza con un motivo.

La operación se registra en los archivos JSON; después, la API devuelve el resultado para la vista correspondiente.

5.3 Mecanismo de persistencia

La aplicación usa archivos JSON locales en una carpeta data/ situada por defecto en la raíz del proyecto. Los archivos principales son pedidos.json, cambios.json, notificaciones.json y configuracion.json. El servidor imprime la ruta efectiva al iniciar y admite PEDIDOSFLOW_DATOS para indicar otra carpeta. El modo de demostración está activo por defecto: crea cinco pedidos y una solicitud pendiente solo cuando falta el conjunto inicial o el archivo de pedidos está vacío; no vuelve a duplicar ni sobrescribir los cambios guardados.

La transcripción reciente del chat usa sessionStorage del navegador y se limita a 120 mensajes. Se restaura al recargar la misma pestaña, pero no sustituye la persistencia del servidor y no guarda imágenes base64 dentro de esa transcripción. Las decisiones de pedidos y solicitudes pendientes se conservan en JSON.

5.4 Tecnologías seleccionadas

- Node.js y TypeScript para el servidor.
- Express para la API REST.
- HTML, CSS y JavaScript en el navegador para el chat y la plataforma del encargado.
- Archivos JSON para persistencia local.
- node:test para pruebas automatizadas; Playwright/Chromium para el recorrido E2E opcional.

- Git y GitHub para control de versiones, tablero, ramas e integración.
- Windows y Visual Studio Code como entorno de desarrollo previsto.

5.5 Decisiones técnicas relevantes

Separar la lógica de reglas del servidor y de la interfaz facilita probar condiciones sin depender de una pantalla.

Mantener la matriz de reglas como configuración editable permite modificar los criterios sin cambiar código.

Derivar la etapa del checklist reduce el riesgo de que la validación use una etapa manual desactualizada.

Usar JSON mantiene baja la complejidad del prototipo académico, aunque no sustituye una base de datos para despliegues con concurrencia o múltiples usuarios.

Conservar separadas la decisión automática y la decisión manual mantiene la trazabilidad.

No se presenta una integración real con WhatsApp Business, autenticación administrativa ni analítica histórica como funcionalidades terminadas: se mantienen en el backlog posterior. El aviso de aprobación o rechazo después de revisión funciona en el chat web simulado; el envío saliente por WhatsApp real requiere configurar la API oficial, credenciales y webhook.

6. Incremento funcional

6.1 Alcance incluido

El paquete ejecutable contiene un flujo vertical verificable: el cliente consulta o modifica un pedido; el servidor valida la entrada y la etapa; el motor aplica la regla o genera una solicitud pendiente; el resultado se guarda en JSON; la plataforma permite consultar el estado actualizado y el historial. Para el camino de revisión, el cliente recibe el aviso definitivo después de la decisión guardada por el encargado.

6.2 Mejoras concretas del segundo incremento

Tabla 8

Mejoras incorporadas en el incremento 2

Mejora	Comportamiento
Ruta de datos estable	La carpeta de datos se resuelve respecto a la raíz del proyecto y se imprime en consola al iniciar.
Escritura JSON reforzada	Se escribe primero en un temporal único y después se reemplaza el archivo final; hay una ruta de recuperación para determinados fallos en Windows.
Entrada libre en el chat	Se conserva la conversación guiada por botones y se añade escritura libre en pasos pertinentes.
Reconocimiento limitado de intenciones	Se reconocen frases comunes para crear, consultar, listar y cambiar datos de un pedido.
Fechas y horas cotidianas	Se interpretan formatos conocidos (por ejemplo, “mañana”, “7 de noviembre” o “3:30 pm”) y se validan antes de enviar la solicitud.
Confirmación explícita	La creación exige una afirmación; las negaciones y respuestas ambiguas no deben generar un pedido.
Transcripción de la pestaña	Se restauran hasta 120 mensajes en sessionStorage, sin guardar cadenas base64 de fotos en el historial local.
Menú compacto de opciones	Las opciones rápidas se sitúan bajo la conversación y separadas del campo de escritura; “Volver al menú” forma parte del mismo panel.
Aviso después de revisión humana	El chat informa primero que la solicitud está pendiente. Solo después de guardar aprobación/rechazo, consulta el pedido y muestra el resultado final.
Precarga estable para demostración	La primera ejecución carga cinco pedidos (#152–#156) y una solicitud de diseño pendiente cuando la carpeta está nueva o vacía. En reinicios posteriores no duplica ni revierte los cambios.

6.3 Instrucciones de ejecución

Requisitos: Node.js 20 o superior y npm. En Windows, el modo recomendado es ejecutar INICIAR_PEDIDOSFLOW.bat desde la carpeta que contiene simultáneamente src\index.ts y package.json del servidor. Si la terminal está en la carpeta exterior donde se descomprimió PedidosFlow-incremento2-documentacion-actualizada, entra primero en la carpeta interior PedidosFlow-incremento2. Antes de ejecutar los comandos, npm run debe

identificar el paquete como pedidosflow@1.0.0. Si muestra figma-make-app@1.0.0 o dice que falta el script verify, estás usando el package.json del prototipo visual y no el del servidor funcional.

```
npm ci
npm run verify
npm start
```

Luego abrir http://localhost:3001. Para usar una carpeta de datos alternativa en PowerShell:

```
$env:PEDIDOSFLOW_DATOS = "C:\PedidosFlowDatos"
npm start
```

Para ejecutar el recorrido E2E con navegador, instalar Playwright y Chromium según docs/PRUEBAS.md, iniciar el servidor con una carpeta de datos de prueba y ejecutar node e2e/recorrido.cjs desde otra terminal. La evidencia debe registrar la salida real. No se deben borrar carpetas antiguas de datos sin comprobar primero cuál contiene los pedidos que se desean conservar.

6.4 Guion de demostración de cinco minutos

Tabla 9

Guion sugerido para la demostración

Tiempo	Demostración
0:00-0:30	Presentar el problema, el repositorio y las funciones que siguen fuera de alcance.
0:30-1:20	Mostrar el chat y consultar el pedido #154 por número o con una frase escrita.
1:20-2:10	Solicitar un cambio de cantidad o fecha; mostrar cómo la etapa y la matriz deciden la respuesta.
2:10-2:50	Enviar un cambio de diseño personalizado y mostrar la cola para revisión humana.
2:50-3:35	Consultar el historial, la etapa de producción y los datos del pedido desde el panel.
3:35-4:15	Crear un pedido de prueba, llegar al resumen y responder “no quiero confirmar”; demostrar que no se registra.
4:15-5:00	Mostrar en la consola la ruta de datos, reiniciar el servidor, consultar la persistencia y terminar con las pruebas y las limitaciones.

7. Evidencias de trabajo colaborativo

El equipo informa que el compañero revisó localmente el proyecto ejecutable y lo consideró satisfactorio. Esa revisión informal se incorpora como antecedente del trabajo colaborativo; para que la rúbrica pueda verificarla, debe quedar registrada también en GitHub sobre la versión exacta del código que se entrega. El pull request #24 abierto en este momento corresponde a documentación, no a la integración del servidor funcional del ZIP. Repositorio: https://github.com/isabel097/pedidosFlow. Tablero: https://github.com/users/isabel097/projects/1. Pull request documental #24: https://github.com/isabel097/pedidosFlow/pull/24. Issues de cierre: #25 sincronización del código ejecutable, #26 regresión y evidencias, y #27 validación de reglas con un negocio real. La rama feature/incremento-2-chat-persistencia existe, pero contiene por ahora cambios documentales; no equivale a que el código del servidor ya esté integrado en main.

Tabla 10

Evidencias requeridas y acciones de cierre

Evidencia	Preparado en el paquete	Pendiente de comprobación o ejecución
Participación de integrantes	El equipo reporta una revisión funcional informal por parte del compañero y el repositorio conserva actividad y PRs anteriores.	Añadir captura de las contribuciones de ambos al incremento 2 y registrar la revisión del código en el PR correspondiente, sin atribuir autores sin comprobarlos.
Ramas de trabajo	Existe feature/incremento-2-chat-persistencia, creada para documentar el incremento; todavía no contiene el servidor del ZIP.	Publicar allí el código ejecutable exacto y capturar el árbol de archivos de esa rama.

Evidencia	Preparado en el paquete	Pendiente de comprobación o ejecución
Commits descriptivos	La rama actual tiene commits documentales; también existen commits descriptivos históricos de incrementos anteriores.	Insertar captura de los commits del código del incremento 2 y verificar los autores con el equipo.
Issues/tareas	Están abiertos los issues #25 para sincronizar el código, #26 para regresión/evidencias y #27 para validación de campo, además de issues históricos.	Asignar responsables reales y actualizar el estado solo a partir de las pruebas, revisión y validación ejecutadas.
Pull request/revisión	Está abierto el PR #24 para cambios de documentación; no es el PR de la implementación ejecutable.	Crear o actualizar el PR del código tras sincronizarlo. Solicitar revisión del compañero y capturar la revisión real de GitHub.
Integración en main	El PR #24 está abierto y no fusionado; el código ejecutable todavía no se ha sincronizado con la rama principal.	Fusionar el incremento funcional solo después de que el código exacto esté publicado, las verificaciones pasen y la revisión esté registrada.

7.1 Evidencias visuales del incremento funcional

Figura 3

Chat con menú compacto y campo de escritura separado

[INSERTAR CAPTURA REAL AQUÍ]

Debe verse el historial de conversación, las opciones rápidas debajo de los mensajes y el área de escritura independiente.

Nota. Sustituir este recuadro por una captura tomada de la versión ejecutada; recortar datos personales reales si los hubiera.

Figura 4

Solicitud escrita y cambio reflejado en el pedido

[INSERTAR CAPTURA REAL AQUÍ]

Mostrar el mensaje escrito por el cliente y, en la vista del encargado o en la consulta posterior, el nuevo valor realmente guardado.

Nota. Sustituir este recuadro por una captura tomada de la versión ejecutada; recortar datos personales reales si los hubiera.

Figura 5

Solicitud pendiente y aviso posterior a la decisión humana

[INSERTAR CAPTURA REAL AQUÍ]

Incluir evidencia de la cola Cambios pendientes y de la respuesta final en el chat después de que el encargado aprueba o rechaza.

Nota. Sustituir este recuadro por una captura tomada de la versión ejecutada; recortar datos personales reales si los hubiera.

Figura 6

Persistencia después de reiniciar el servidor

[INSERTAR CAPTURA REAL AQUÍ]

Mostrar la ruta data/ informada por consola y los datos del mismo pedido antes/después del reinicio, sin restablecer la modificación.

Nota. Sustituir este recuadro por una captura tomada de la versión ejecutada; recortar datos personales reales si los hubiera.

Figura 7

Rama y pull request del incremento 2

[INSERTAR CAPTURA REAL AQUÍ]

Capturar la rama del incremento 2, el pull request hacia main y su revisión real. No declarar aprobado o fusionado hasta que GitHub lo muestre.

Nota. Sustituir este recuadro por una captura tomada de la versión ejecutada; recortar datos personales reales si los hubiera.

Figura 8

Salida real de la verificación automatizada

[INSERTAR CAPTURA REAL AQUÍ]

Pegar la captura de npm run verify y, si aplica, del recorrido E2E. El resultado de esta versión se debe copiar exactamente de la terminal.

Nota. Sustituir este recuadro por una captura tomada de la versión ejecutada; recortar datos personales reales si los hubiera.

7.2 Secuencia para la publicación y revisión del incremento 2

Actualizar main con los cambios que el equipo haya decidido conservar y crear la rama feature/incremento-2-chat-persistencia.

Crear issues para ruta de datos, escritura JSON, entrada libre, intenciones, validación de fecha/hora, confirmación y transcripción.

Hacer commits pequeños y descriptivos; por ejemplo: “fix: estabiliza ruta de datos (PF2-01)” o “test: agrega escenarios de conversación (PF2-04)”. Estos son ejemplos, no commits ya existentes.

Abrir un pull request hacia main con instrucciones de prueba y evidencia. La otra persona del equipo debe revisar el código y solicitar cambios si corresponde.

Tras la aprobación y pruebas correctas, fusionar el pull request y guardar capturas del PR, los checks, la rama main y el tablero actualizado.

8. Pruebas y criterios de calidad

8.1 Estrategia de pruebas

La estrategia combina pruebas unitarias del motor de reglas, pruebas de API y persistencia, recorrido E2E del navegador y comprobaciones estáticas. La documentación anterior registró resultados históricos para una versión base (80 pruebas automatizadas y un

recorrido E2E anterior); esas cifras no deben atribuirse automáticamente a esta versión. Para la copia final se debe adjuntar la salida real de npm run verify y el resultado del recorrido del navegador. La organización de los atributos de calidad y sus criterios de evaluación también toma como referencia el modelo de calidad de producto de ISO/IEC 25010:2023, sin implicar certificación ni conformidad formal del sistema (International Organization for Standardization [ISO], 2023).

En la revisión técnica de preparación, node --check terminó correctamente para public/chat.js, public/app.js, public/util.js y e2e/recorrido.cjs. En la comprobación reportada el 22 de octubre, npm run se ejecutó desde la raíz funcional y mostró el script verify; después, npm ci falló con EPERM al intentar eliminar esbuild.exe. Los errores posteriores indican una instalación incompleta: tsc no encontró express y npm start no encontró tsx. El package.json funcional define verify = npm run typecheck && npm test; la suite completa debe repetirse después de limpiar node_modules e instalar las dependencias correctamente. Insertar en la Figura 8 la salida real de la verificación final.

8.2 Casos de prueba y resultados

Tabla 11

Escenarios de prueba relevantes

ID	Escenario	Resultado esperado	Estado de evidencia
CP-01	Cambio de cantidad permitido en etapa temprana.	Pedido actualizado e historial creado.	Escenario definido en la suite; adjuntar salida real de regresión de esta versión.
CP-02	Cambio en producción que requiere revisión.	Pedido no cambia antes de la decisión humana.	Escenario definido en la suite; adjuntar salida real de regresión de esta versión.

ID	Escenario	Resultado esperado	Estado de evidencia
CP-03	Cambio bloqueado en control final.	Rechazo con motivo; pedido sin modificar.	Escenario definido en la suite; adjuntar salida real de regresión de esta versión.
CP-04	Pedido inexistente.	Respuesta controlada; no se registra cambio.	Escenario definido en la suite; adjuntar salida real de regresión de esta versión.
CP-05	Reiniciar el servidor después de guardar un cambio.	Pedidos e historial conservados.	Escenario definido en la suite; adjuntar salida real de regresión de esta versión.
CP-06	Enviar diseño personalizado con foto.	Solicitud pendiente para revisión del encargado.	Escenario definido en la suite; adjuntar salida real de regresión de esta versión.
CP-07	Editar una regla de configuración.	La siguiente validación aplica la nueva regla.	Escenario definido en la suite; adjuntar salida real de regresión de esta versión.
E2E-8a	Escribir “quiero cambiar la fecha del pedido #154”.	Identifica el pedido e inicia pregunta por la nueva fecha.	Recorrido documentado; ejecución/captura final por registrar.
E2E-8c	Escribir “quiero consultar el pedido #154”.	Consulta y muestra datos guardados del pedido.	Recorrido documentado; ejecución/captura final por registrar.
E2E-8d	Recargar la pestaña tras enviar mensajes.	Transcripción reciente sigue visible en la misma pestaña.	Recorrido documentado; ejecución/captura final por registrar.

ID	Escenario	Resultado esperado	Estado de evidencia
E2E-8e	Responder “no quiero confirmar” en un resumen.	No crea ni confirma un pedido nuevo.	Recorrido documentado; ejecución/captura final por registrar.
E2E-9	Revisar consola durante el recorrido completo.	No aparecen errores nuevos de JavaScript.	Recorrido documentado; ejecución/captura final por registrar.

8.3 Errores identificados y ajustes realizados

Tabla 12

Registro de errores y ajustes

Error o problema	Cómo se identificó	Ajuste realizado
La carpeta de datos podía depender de la carpeta actual desde la que se iniciaba el servidor.	Revisión del código de almacenamiento y del arranque.	Resolver la carpeta de datos desde la raíz del proyecto e imprimir la ruta efectiva al iniciar.
La escritura directa del archivo JSON podía dejar un archivo incompleto o competir con otra escritura.	Inspección de almacenamiento y casos de persistencia.	Escritura a un temporal único y reemplazo posterior; recuperación prevista en determinados fallos del sistema de archivos.
El chat dependía en exceso de botones/respuestas predeterminadas.	Revisión de la experiencia conversacional.	Añadir entrada libre y reconocer un conjunto limitado de intenciones frecuentes.
La creación podía avanzar con una respuesta negativa que contuviera la palabra “confirmar”.	Revisión de la lógica de confirmación.	Dar prioridad a negaciones conocidas y no crear el pedido sin confirmación afirmativa.
La transcripción se perdía al recargar la pestaña.	Revisión de la continuidad del chat.	Restaurar la transcripción reciente desde sessionStorage, sin persistir las imágenes base64 en esa copia local.

8.4 Criterios de aceptación cumplidos

El código del paquete incluye las correcciones descritas en las Tablas 8 y 12 y los archivos JavaScript principales superaron la comprobación sintáctica. Además, la comprobación inicial con npm run confirmó que el paquete funcional define el comando verify.

En el intento de verificación del 22 de octubre de 2026, npm ci no pudo retirar esbuild.exe (error EPERM), por lo que la instalación quedó incompleta: tsc no encontró express y npm start no encontró tsx. Este resultado deja registrada una comprobación real con bloqueo de entorno, pero no equivale a una regresión aprobada. Los criterios se separan entre comportamiento incorporado y verificación respaldada por una ejecución que finalice correctamente.

Tabla 13

Estado de criterios del segundo incremento

Criterio	Estado en código	Verificación faltante
Ruta de datos y archivos estáticos	Implementado en el código	La ruta estable está incorporada; insertar captura de consola con la ruta efectiva (Figura 6).
Escritura segura y datos tras reinicio	Implementado en el código	La lógica está incorporada; adjuntar el resultado de prueba de reinicio y consulta (Figura 6).
Reconocimiento de intención / consulta de pedido	Implementado en el chat	La corrección está incluida en el paquete; adjuntar resultado E2E o captura real (Figuras 4 y 8).
Fecha y hora y validación	Implementado en el chat	La corrección está incluida en el paquete; adjuntar resultado E2E o captura real (Figuras 4 y 8).
No crear pedido ante negación	Corregido en el código	La corrección está incluida en el paquete; adjuntar resultado E2E o captura real (Figuras 4 y 8).
Transcripción tras recarga	Implementada con sessionStorage	La corrección está incluida en el paquete; adjuntar resultado E2E o captura real (Figuras 4 y 8).

Criterio	Estado en código	Verificación faltante
Diseño visual existente	Clases y estructura conservadas; se añade campo de texto	La nueva distribución está incluida en el ZIP; insertar captura real del chat (Figura 3).

8.5 Definition of Done

La historia cumple sus criterios de aceptación definidos.

Confirmar que la instalación de dependencias termina sin errores y que npm run verify finaliza correctamente.

La funcionalidad se recorre de punta a punta en la interfaz y los casos alternativos y de error tienen evidencia.

Los datos funcionales permanecen después de cerrar y volver a abrir el servidor.

El cambio es revisado por la otra persona del equipo e integrado en main mediante pull request aprobado.

README y documentos describen las instrucciones reales de ejecución, decisiones técnicas y limitaciones.

Estado de la Definition of Done: parcialmente verificada. Se comprobó que el paquete funcional declara el script verify y los archivos JavaScript principales superaron la comprobación sintáctica. La instalación de dependencias quedó bloqueada por EPERM, por lo que faltan la instalación limpia, la salida satisfactoria de la regresión, las capturas funcionales solicitadas y la revisión cruzada del pull request del incremento 2.

8.6 Limitaciones actuales

- El chat es una simulación web; no existe conexión activa con WhatsApp Business Platform.
- La interpretación de texto usa reglas y frases conocidas; no es comprensión abierta de cualquier mensaje.

- El almacenamiento es local en JSON y requiere definir una estrategia distinta si se despliega con varias instancias o escrituras concurrentes.
- Las reglas de producción deben validarse con una persona encargada de un negocio real.
- No se incluye autenticación administrativa en la versión actual.

La suite de regresión y los nuevos escenarios E2E están incluidos en el paquete, pero deben ejecutarse en un entorno con dependencias completas; los resultados de ejecución se documentarán con la salida real de la terminal y capturas del navegador.

Las evidencias de la rama y el pull request del incremento 2 deben añadirse después de publicar los archivos actuales. La revisión por otra persona y el merge en main se consideran completados únicamente cuando GitHub los muestre como tales.

Conclusión

PedidosFlow conecta solicitud, validación, decisión, persistencia y consulta. El segundo incremento mejora la estabilidad del almacenamiento, permite redactar solicitudes comunes con palabras propias, organiza el menú del chat y retiene los avisos de aprobación o rechazo hasta que el encargado guarda una decisión. Las excepciones que requieren criterio humano no modifican el pedido antes de la revisión.

La entrega se considerará cerrada cuando el código y la documentación estén sincronizados en el repositorio, se ejecute la regresión de la versión final, se incorporen las capturas reales en los espacios marcados y el pull request tenga revisión cruzada e integración verificables. La integración con WhatsApp real, la autenticación y las métricas siguen fuera del alcance de este incremento.

Referencias

American Psychological Association. (2020). *Publication manual of the American Psychological Association* (7th ed.). American Psychological Association.

Azionya, C. M., & Mashigo, R. D. (2025). WhatsApp Business as a digital engagement tool: Factors influencing success among South African small and micro-retailers. Communitas, 30, 1–16. https://doi.org/10.38140/com.v30i.9321

Cohn, M. (2004). *User stories applied: For agile software development.* Addison-Wesley.

Galvis Gómez, M. L., Moreno Ibáñez, A. M., Rueda Peña, M. P., & Ruiz Rico, D. (2020). Pandemia y emprendimiento: Relatos de experiencias en Bucaramanga [Trabajo de grado, Universidad Autónoma de Bucaramanga]. Repositorio Institucional UNAB. https://repository.unab.edu.co/bitstream/handle/20.500.12749/12467/2020_Tesis_Maria_Lucia_Galvis_Gomez.pdf?sequence=1

GitHub. (s. f.). *GitHub flow*. GitHub Docs. https://docs.github.com/en/get-started/using-github/github-flow

International Organization for Standardization. (2023). *ISO/IEC 25010:2023 Systems and software engineering—Systems and software quality requirements and evaluation (SQuaRE)—Product quality model*. https://www.iso.org/standard/78176.html

Meta. (2025, April 3). Ways to manage your businesses chats on WhatsApp. https://about.fb.com/news/2025/04/ways-to-manage-your-businesses-chats-on-whatsapp/

Schwaber, K., & Sutherland, J. (2020). *The Scrum Guide: The definitive guide to Scrum: The rules of the game.* https://scrumguides.org/scrum-guide.html

Wang, Y., & Tseng, M. M. (2014). Incorporating tolerances of customers' requirements for customized products. *CIRP Annals, 63*(1), 129–132. https://www.researchgate.net/publication/261953984_Incorporating_tolerances_of_customers_requirements_for_customized_products

Wassenger. (2025, 20 de febrero). Cómo una panadería local automatizó los pedidos de clientes vía WhatsApp con Wassenger. https://wassenger.com/blog/es/como-una-panaderia-local-automatizo-los-pedidos-de-clientes-via-whatsapp

Zhang, Q., & Tseng, M. M. (2009). Modelling and integration of customer flexibility in the order commitment process for high mix low volume production. *International Journal of Production Research, 47*(22), 6397–6416. https://doi.org/10.1080/00207540802266474

Anexo A. Guía breve para validar reglas con una pastelería

Estado: instrumento preparado para documentar de forma estructurada el contraste de las reglas con una persona encargada. Antes de registrar respuestas, explicar el propósito académico y solicitar autorización; después, resumir los hallazgos y su efecto en los criterios del prototipo.

Fecha de consulta: _____ Medio: _____

Tipo de negocio y rol de la persona (sin datos personales innecesarios):

_____

¿Por qué canales reciben pedidos actualmente y cómo identifican cada pedido confirmado?

¿Qué datos necesitan consultar cuando un cliente pide cambiar cantidad, diseño o fecha?

¿Cómo determinan si un cambio de cantidad puede aceptarse según la etapa de producción?

¿Qué señales utilizan para identificar un día saturado o sin capacidad?

¿Qué hacen cuando se solicita una fecha pasada o un cambio que altera la producción ya iniciada?

¿Qué diseños pueden aceptar automáticamente y cuáles requieren revisar con una persona?

¿Cómo registran una aprobación o rechazo y cómo se entera el cliente de la decisión?

Hallazgos confirmados:

_____

Regla de PedidosFlow que debería ajustarse o validarse:

_____

Limitaciones o aspectos que no se pudieron comprobar:

_____